



AI시대, 차세대 클라우드의 새로운 비전과 기술

openKcloud DevDay 2025

AI반도체에서 클라우드까지 옴져버빌리티의 모든 것

이노그리드
김바울 수석





목 차

1. AI 인프라 모니터링의 중요성
2. 모니터링에서 옴서버빌리티로
3. AI 반도체·클라우드 옴서버빌리티 아키텍처
4. AI 반도체 옴서버빌리티 확장 방향



AI반도체에서 클라우드까지 옮겨버릴리티의 모든 것

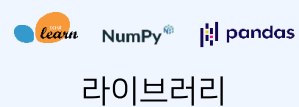
1. AI 인프라 모니터링의 중요성



AI 인프라, 왜 '보이는가'가 더 중요해졌나

- (AI 서비스의 확산) AI 학습 및 서빙 관련 서비스 수요 증가
- (다양한 AI반도체 등장) GPU·NPU 등 AI 서비스 운영을 위한 다양한 장치 개발
- (복잡한 환경 관리 수요) 단순 설치 운영에서 "잘 돌고 있는지, 더 잘 돌릴 수 있는지"에 관심이 증가

AI 학습 및 서빙을 위한 서비스 및 라이브러리



클라우드 서비스



Google Cloud



Microsoft Azure



openstack.

AI 반도체 인프라



CPU



GPU



NPU



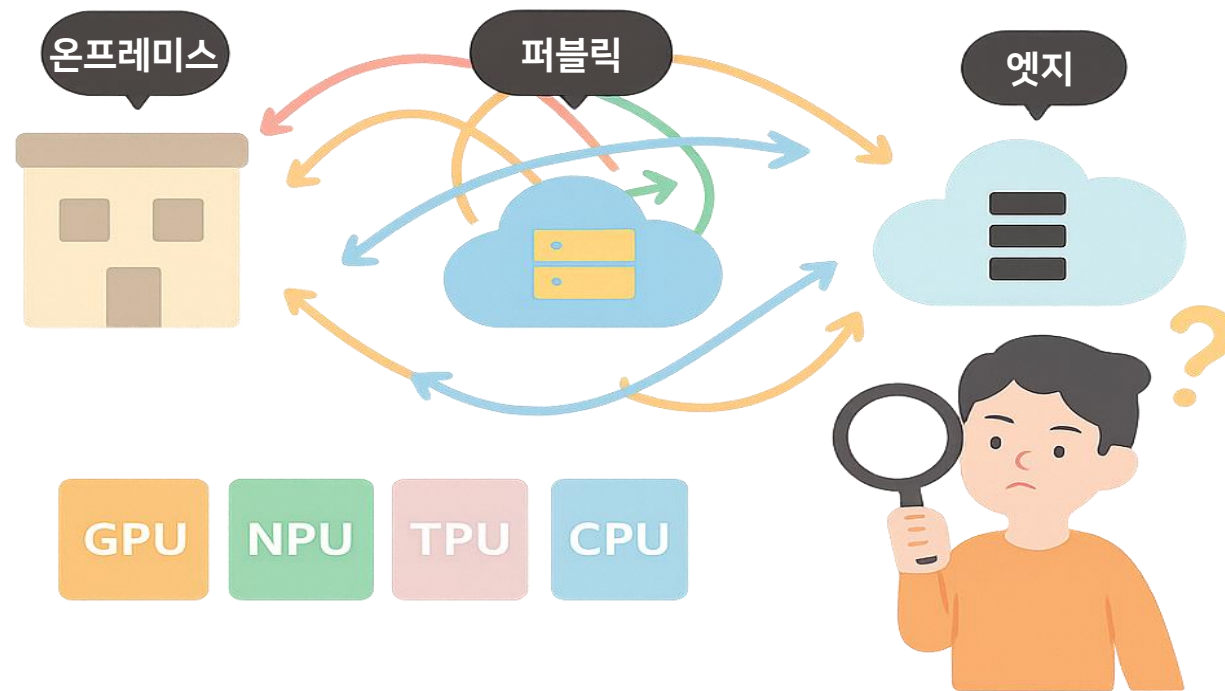
TPU





클라우드에서 엣지까지: 운영 복잡도 폭증

- (하이브리드 구성 증가) 온프레미스 + 퍼블릭 클라우드 + 엣지(마이크로DC) 등 혼합 구조의 확산
- (인프라 복잡성 증가) 워크로드가 어느 리전/노드/칩에서 도는지 파악이 어려움
- (원인 분석의 어려움) 장애·성능·비용 이슈의 원인이 여러 레이어에 흩어지는 상황



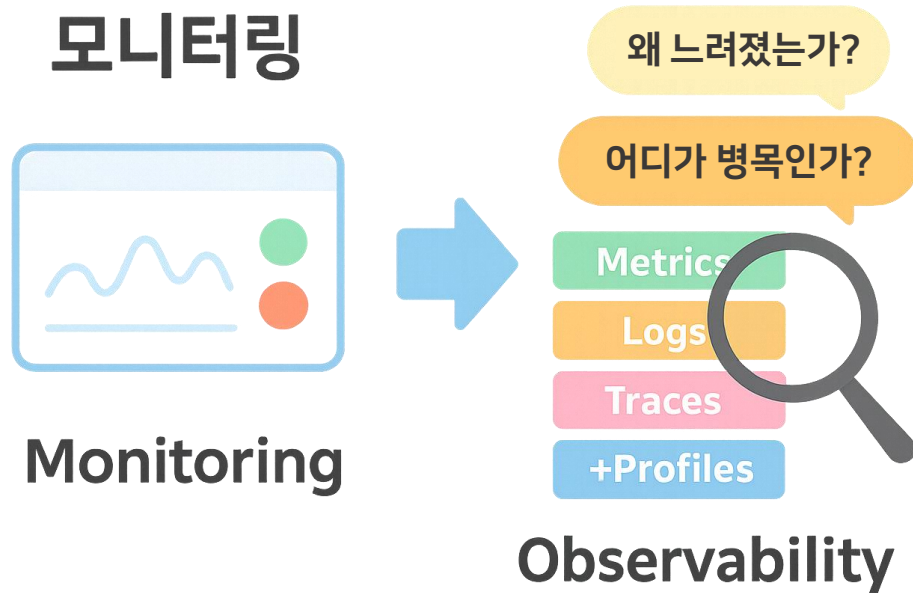
시반도체에서 클라우드까지 오피버빌리티의 모든 것

2. 모니터링에서 오피버빌리티로



모니터링 vs 옵저버빌리티

- 모니터링(Monitoring): 미리 정한 지표/대시보드로 상태 체크 & 알람
- 옵저버빌리티(Observability): 시스템 메트릭, 로그, 추적 정보를 기반으로 원인을 상세하게 추론





모니터링에서 옴저버빌리티로의 변화

시대	주요 특징	핵심 도구/서비스	모니터링 대상	한계점
전통적 모니터링 시대 (~2015)	- Reactive 접근 - 사전 정의된 메트릭 - 임계값 기반 알림 - 데이터 사일로	- Nagios - Zabbix - SNMP - 기본 로그 수집	- CPU, 메모리, 디스크 - 네트워크 대역폭 - 단순 애플리케이션 메트릭	- 분산 시스템 부적합 - 수동 상관관계 분석 - "무엇"만 알 수 있음
컨테이너/클라우드 시대 (2016~2019)	- 마이크로서비스 등장 - 컨테이너 오케스트레이션 - 메트릭 중심화 - 시계열 데이터 중요성	- Prometheus - Grafana - Kubernetes Metrics	- 컨테이너 리소스 - Pod/Service 상태 - 클러스터 메트릭	- 로그-메트릭 분리 - 추적 기능 부족 - GPU 가시성 제한
GPU 모니터링 (2017~2021)	- AI/ML 워크로드 증가 - GPU 전용 메트릭 - 데이터센터 GPU 관리 - Exporter 패턴 정착	- NVIDIA DCGM - dcgm-exporter - nvidia-smi	- GPU 사용률/메모리 - 온도/전력 - NVLink 대역폭 등	- NVIDIA 종속성 - 애플리케이션 연관성 약함 - 멀티벤더 불가
통합 옴저버빌리티 시대 (2020~2023)	- 3 Pillars 통합 - Distributed Tracing - 컨텍스트 상관관계 - OpenTelemetry 표준화	- OpenTelemetry - Jaeger/Zipkin - Elastic Stack (ELK)	- Metrics (메트릭) - Logs (로그) - Traces (추적)	- 고비용 구조 - 데이터 볼륨 폭증 - 복잡한 설정
AI 인프라 옴저버빌리티 (2023~현재)	- NPU/TPU 등장 - AI 워크로드 특화	- DCGM 4.x - TPU Monitoring Library - AMD ROCm 7.0	- NPU: TOPS, 메모리 대역폭 - TPU: Tensor Core 활용률, HBM - Multi-accelerator 토폴로지	- 표준화 부족 - 벤더별 도구 단편화 - 통합 가시성 어려움



AI반도체 기반 옴저버빌리티로의 필요성

단계	접근 방식	질문 유형	데이터 타입	가속기 지원
1단계: Monitoring	Reactive "무엇이 잘못되었나?"	- CPU가 높은가? - 서버가 다운되었나? - 디스크가 찼나?	- Metrics (메트릭) - 사전 정의된 임계값 - 알림	없음 (CPU/메모리만)
2단계: Enhanced Monitoring	Proactive "언제 문제가 발생할까?"	- 어느 서비스가 느린가? - 리소스 경쟁이 있나? - 용량이 부족한가?	- Metrics - Logs (분리됨) - 시계열 데이터	GPU: nvidia-smi (기본) 제한적 가시성
3단계: Observability	Proactive + Exploratory "왜 이런 일이 일어났나?"	- 어떤 요청이 실패했나? - 어디서 지연이 발생했나? - 왜 이 패턴이 나타났나?	- MELT - Metrics - Events - Logs - Traces	GPU: DCGM + dcgm-exporter NVIDIA 중심 추적 부족
4단계: AI Observability	Intelligent + Predictive "앞으로 무슨 일이 일어날까?"	- GPU 병목이 어디인가? - 모델 성능이 저하되나? - NPU 활용률이 최적인가? - TPU 비용을 줄일 수 있나?	- MELT + Profiling - AI 특화 메트릭 - Tensor Core 활용률 - HBM 사용량 - 커널 실행 시간	멀티 가속기: - NVIDIA: DCGM, CUPTI - AMD: ROCm, ROCtracer - Google: TPU Library - Intel: oneAPI, Level Zero

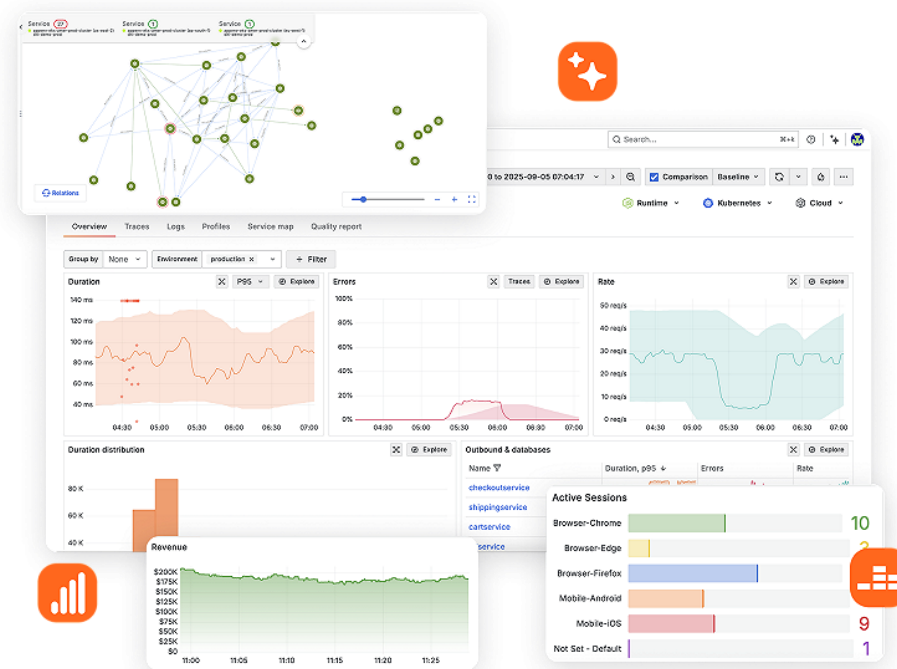
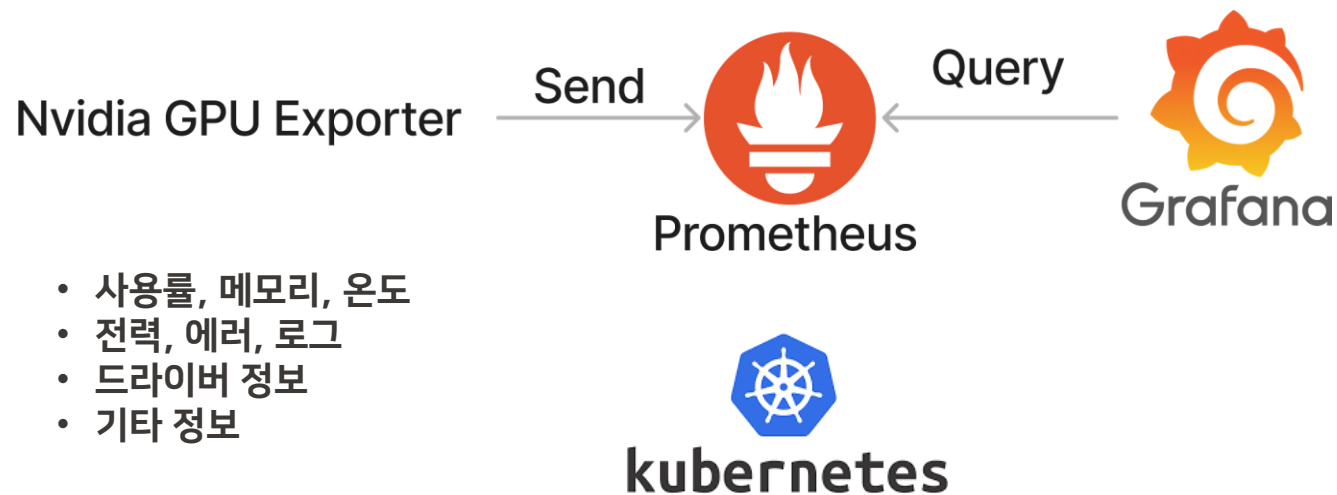
AI반도체에서 클라우드까지 옴져버빌리티의 모든 것

3. AI 반도체·클라우드 옴져버빌리티 아키텍처



전형적인 GPU 모니터링 스택

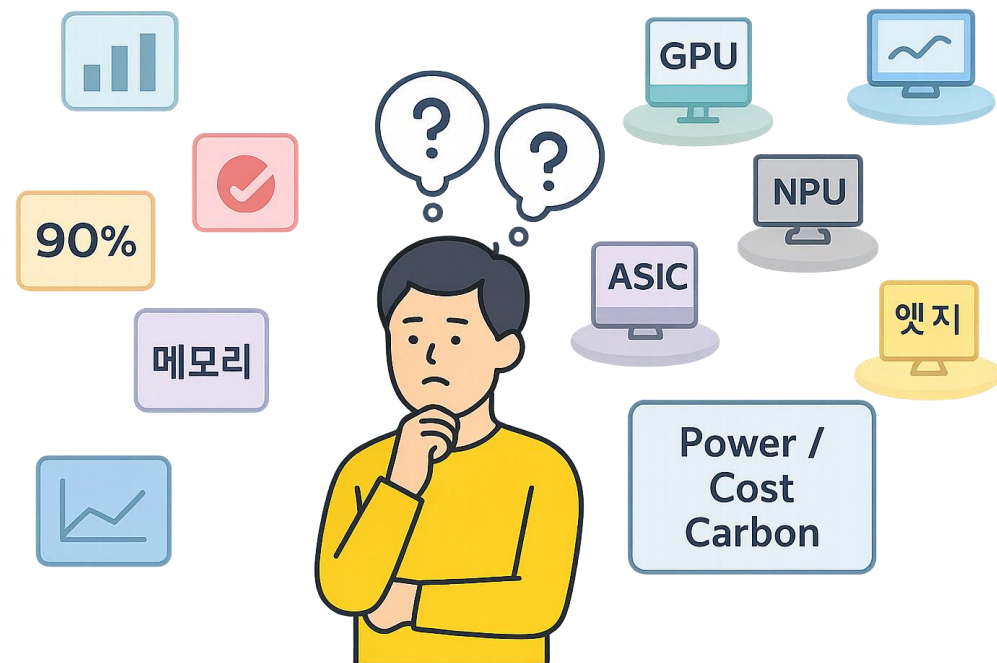
- GPU Exporter → Prometheus → Grafana 구조
- 수집 지표 예:
 - 사용률, 메모리, 온도, 전력, 에러, 로그 등





무엇이 부족한가?

- 지표는 많은데 어느 부분이 실질적으로 영향을 미치는지 확인하기 어려움
- GPU 이외의 NPU·전용 AI칩·엣지 디바이스는 각자 따로 수집
- 질문:
 - “GPU는 90%인데 서비스는 왜 느린가?”
 - “NPU 모니터링 정보는 어떻게 보는가?”
 - “어떤 잡이 전력을 제일 많이 먹는가?”
→ 답을 내기 어려움

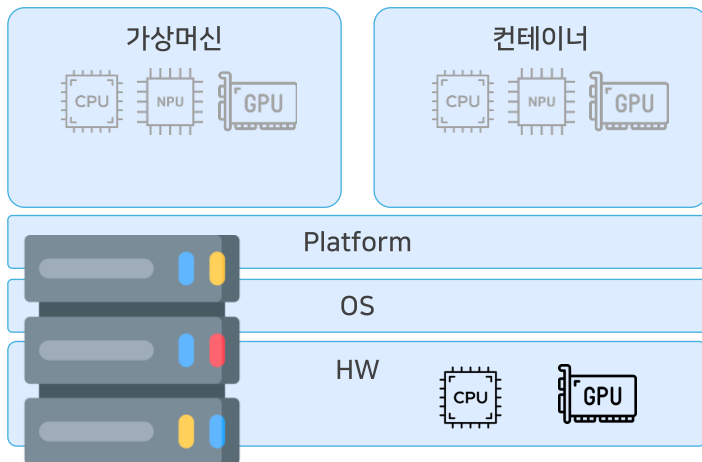




AI반도체 모니터링을 위한 기반 기술

- 물리 서버, AI반도체, 가상 자원까지 모든 정보를 수집하기 위한 구조 설계가 필요

기존의 아키텍처와 호환



AI반도체 클라우드 시스템

AI반도체 모니터링 지원

항목	리벨리온 (Rebellions)	퓨리오사AI (FuriosaAI)	NVIDIA GPU
운영환경 OS	Ubuntu 18.04/20.04 이상, Red Hat(RHEL) 공식 지원	Ubuntu 20.04 LTS 이상 (공식), ARM/Windows: 미지원, CentOS: 확인 불가	Ubuntu, RHEL/CentOS, Windows, MacOS 등 광범위 지원
K8s 지원	K8s Device Plugin 공식 지원	K8s Device Plugin 공식 지원	K8s Device Plugin 공식 지원
SDK/툴 제공	RBLN SDK: 드라이버, 컴파일러, 런타임, 프로파일러, 서빙 프레임워크	Furiosa SDK: Python/C API, Inference Server, 프로파일러, 명령행 도구	CUDA, cuDNN, TensorRT, DeepStream, nvidia-smi, DCGM Exporter, nvidia-top 등
Prometheus 지원	Prometheus Exporter 공식 지원	Prometheus Exporter 공식 지원	DCGM Exporter 통한 Prometheus 공식 지원

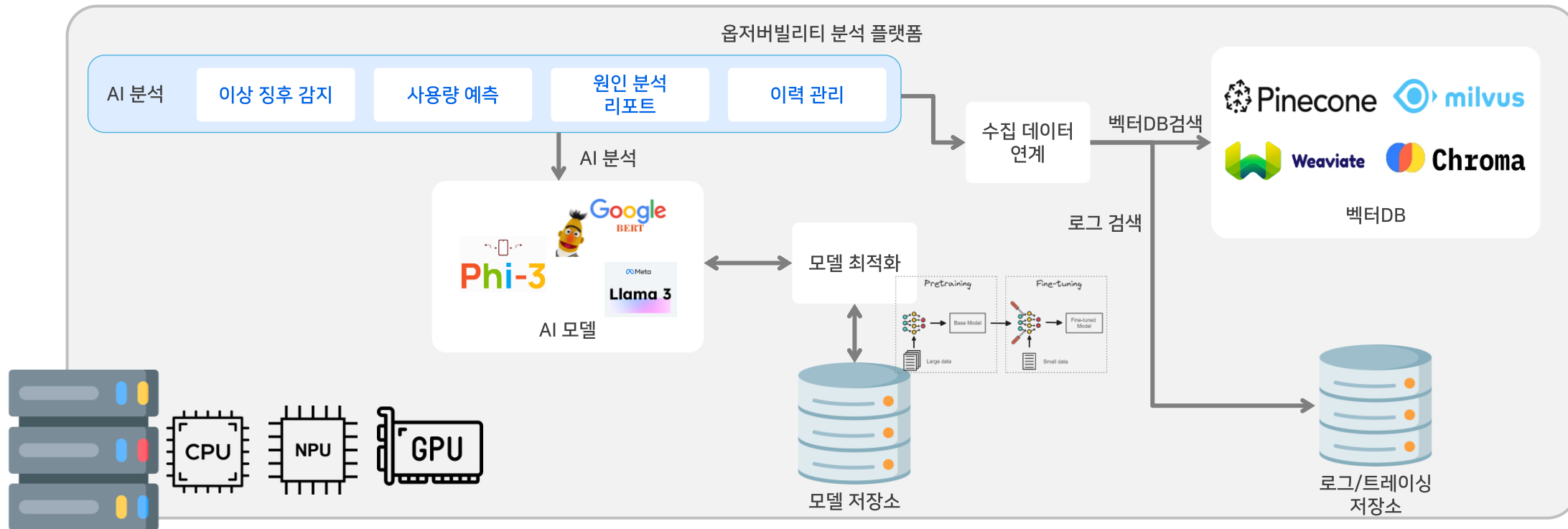
하드웨어/전력 모니터링 지원

항목	Kepler	IPMI
주요 목적	소프트웨어 기반 에너지/전력 모니터링, 특히 Kubernetes 지원	하드웨어 기반 원격 서버 관리 및 전력/장애 관리
측정/활용 범위	CPU/GPU/메모리/노드/컨테이너/Pod별 전력 사용 추정 가능	서버 전체 하드웨어 상태(전력, 온도, 팬, 이벤트 등)
주요 수집 방식	eBPF, RAPL, NVML, ACPI, Redfish, 일부 IPMI 연동 가능	BMC 칩셋 통한 센서 직접 측정 및 실시간 모니터링
활용 환경	클라우드, Kubernetes(컨테이너) 환경 및 Bare Metal	서버 하드웨어(운영체제와 무관), 데이터센터



AI반도체 유통버빌리티를 위한 기술

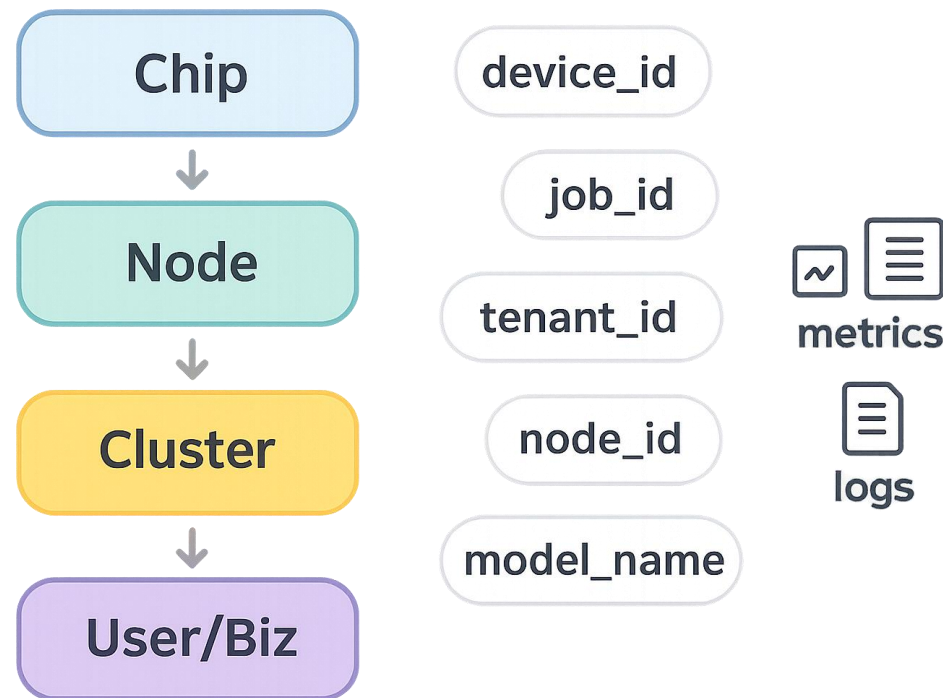
- 다양한 데이터를 통합 분석하기 위한 AI 분석 기술 필요





칩에서 서비스까지: 레이어 모델

- 다양한 구조를 효율적으로 분석하기 위한 레이어 구성
 - Chip → Node → Cluster → Service → User 등
- 단일 지표가 아니라, 다양한 지표를 통합하여 연결하여 분석이 필요
 - 성능: QPS, Latency, Throughput
 - 전력/에너지: W, kWh, 효율(QPS/W 등)
 - 온도/열: Thermal throttling 여부
 - 신뢰성: 에러, 리트라이, 하드웨어 이슈
 - ... 등





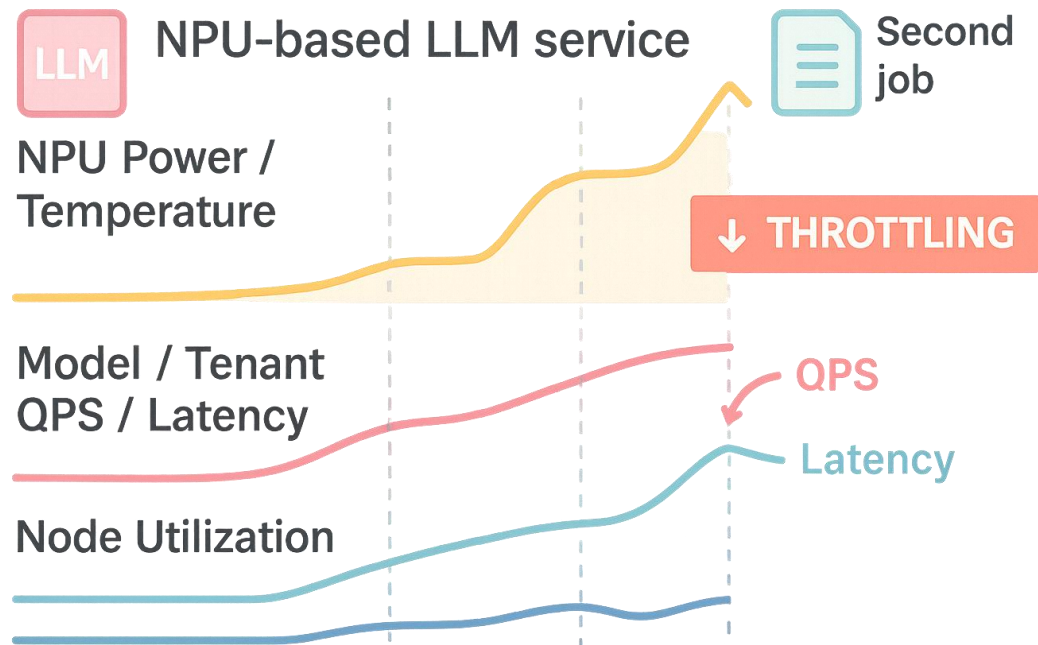
유즈케이스 시나리오: AI 서비스가 느려진 이유 찾기

• 상황 예

- 특정 NPU 기반 LLM 서빙 서비스
- 응답 지연 증가
- 같은 노드에 다른 job이 올라오면서 전력·온도 급상승

• 한 화면에서 보고 싶은 것

- NPU 전력/온도 타임라인
- 해당 시간대 모델/테넌트별 QPS·Latency
- 노드 자원 사용률 및 동시 job 정보

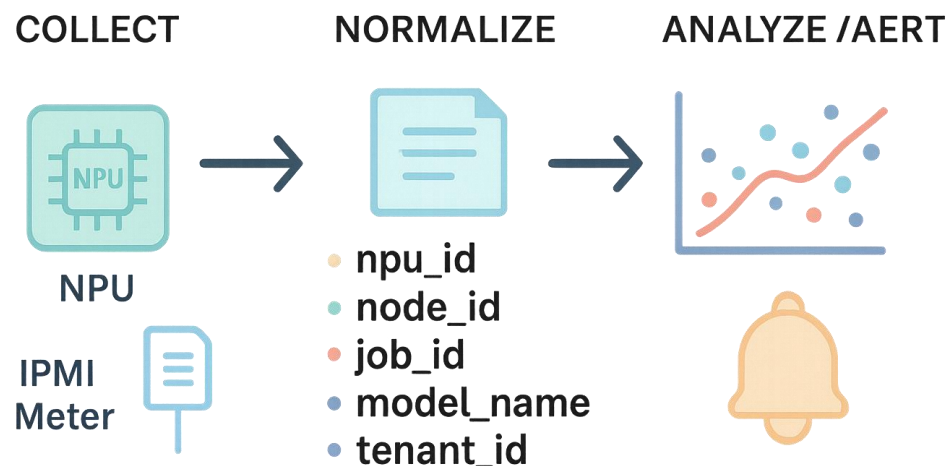




유즈케이스 시나리오: AI 서비스가 느려진 이유 찾기

- 수집
 - NPU Vendor SDK/드라이버 지표
 - IPMI/전력 미터로 노드·랙 단위 전력 수집
- 정규화
 - npu_id, node_id, job_id, model_name, tenant_id 태깅
- 활용
 - 전력(W) vs 처리량(QPS) vs 지연시간(ms) 상관 분석
 - 가장 효율 좋은 NPU/노드/모델 조합 찾기
 - 비효율적인 조합 자동 탐지·알림

NPU + Power Observability Design Points



AI반도체에서 클라우드까지 오피저버빌리티의 모든 것

4. AI반도체 오피저버빌리티 확장 방향



인사이트 활용: 성능·전력·비용 최적화

- 성능: 모델/버전/디바이스별 프로파일링 → 스케줄링·배치 최적화
- 전력/열: 스로틀링 전 징후 감지, 전력 피크 완화 운영 정책 설계
- 비용(FinOps): “어떤 모델/고객이 얼마만큼의 GPU/NPU+전력 비용을 쓰는가?”를 수치화

성능 최적화

AI 서비스, 모델 운영에 대한 성능 분석 및
AI반도체 특화 배치 최적화

전력/열 효율성

스로틀링 예방 및 전력이나 열 효율적인
분산 배치 및 관리

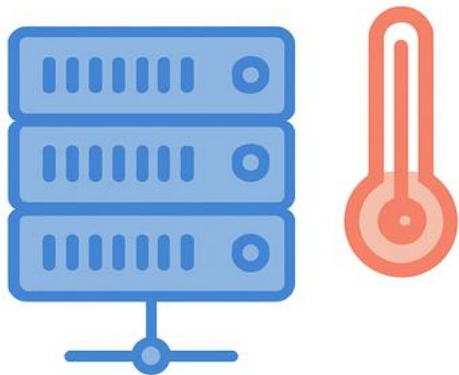
비용 최적화

서비스 효율적인 운영 지원 및
비용 효율적인 운영 지원



확장 방향: 엣지·멀티클라우드·AI Ops

- 엣지/마이크로DC: 전력·공간·냉각 제약 → 전력·온도 지표의 중요성 극대화
- 멀티클라우드: 클라우드/벤더별 지표 체계를 공통 스키마로 흡수하는 계층 레이어
- AI Ops: 수집된 메트릭/로그/트레이스를 기반으로 자동 스케일링, 리소스 재배치, 자율 치유까지 확장



EDGE / MICRO DC



MULTI-CLOUD



AUTONOMOUS



맺음말

**AI반도체에서 클라우드까지 하나의 관측 뷰를 만들면,
성능·전력·비용·품질을 동시에 최적화할 수 있다.**

감사합니다.

